

## EE446 Lab 8 report

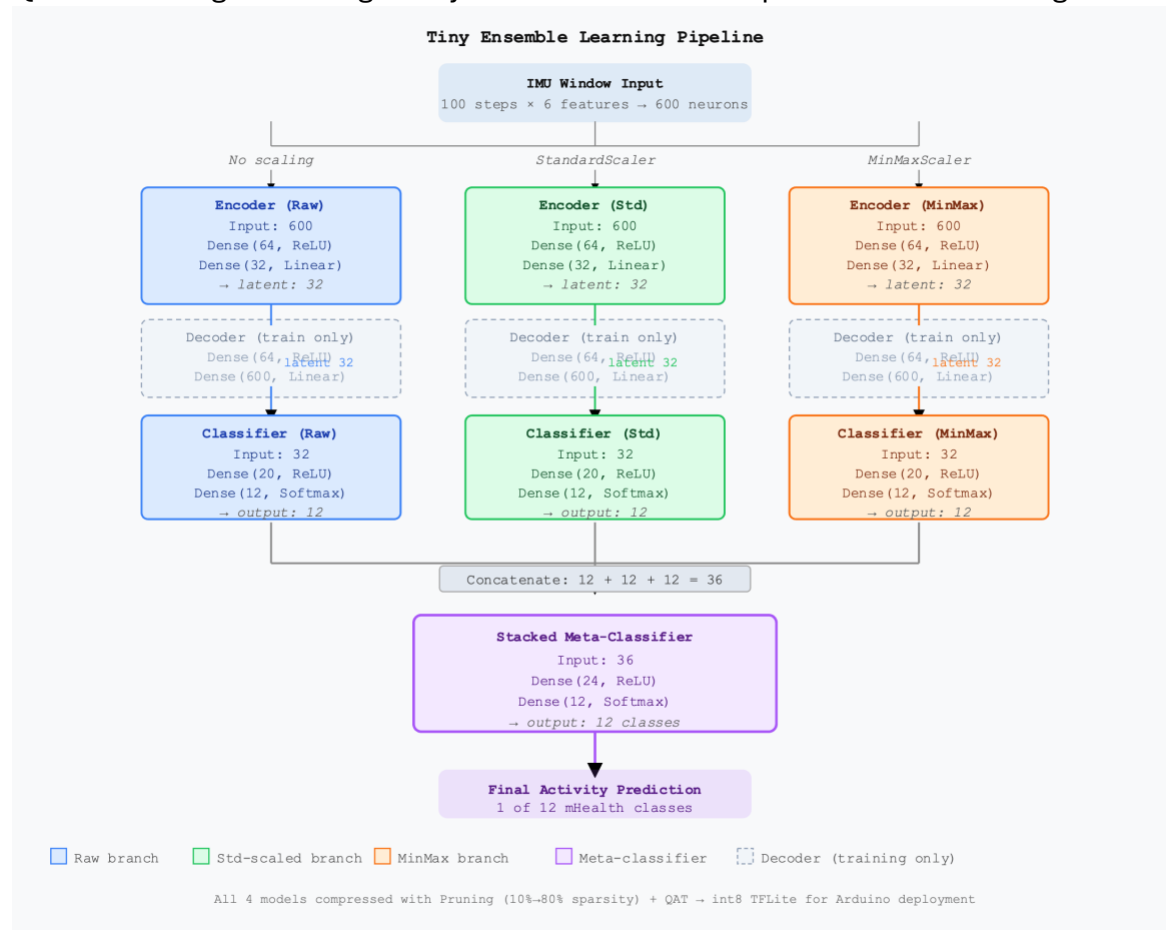
Part 1: <https://studio.edgeimpulse.com/studio/1015066>

Screen shot showing example of 50/50 between idle and lift and example of .996 probability of lift once I lifted it. For some reason the output when idle just says .5 and .5 but it always detects lifts with high confidence.

```
Classification predictions:
  idle: 0.179687
  lift: 0.820312
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 162 ms, inference 6 ms, anomaly 0 ms, postprocessing
Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 158 ms, inference 6 ms, anomaly 0 ms, postprocessing
Classification predictions:
  idle: 0.500000
  lift: 0.500000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 166 ms, inference 577 us, anomaly 0 ms, postprocessing
```

## Part 2:

Question 1 diagram: Designed by me and then created polished version using Claude:



## Question 2:

The pruning is applied using PolynomialDecay schedule which ramps up the sparsity from 10% to 80% over 500 steps across 5 epochs. This dramatically compresses the model. The `prune_low_magnitude` wrapper zeroes the smallest magnitude weights.

The pruning wrappers and QAT wrappers are both wrapping the same keras layers. This means that applying `quantize_model` on top of the pruned layers the two will conflict. `Strip_pruning` fixes this by first cleaning the dense model which allows the QAT wrapper to work without interference.

The mask-enforcement is needed because the QAT's gradient updates will push zeroed weights away from zero which cancels out the pruning. `MaskEnforcerCallback` fixes this by using a binary mask after each training batch and epoch which keeps zeroed weights at zero.

The converter is `TFLITE_BUILTINS_INT8` with `inference_input_type` and `inference_output_type` both set to `int8`. Representative data is required for full integer quantization to observe the actual range of the activations between layers. Without it, the quantization can correctly map the float activations to 8-bit ints.

The pruning and quantization allow the model to be small enough to fit on the tight flash space constraints of the Arduino ble. \*0% sparsity dramatically reduces the size of the model and from there there is also quantization which reduces it even more. The end result is a model that is less than 1/10<sup>th</sup> the size of the original.

### Question 3.

```
58 ms, inf
on predict
000
000
rencing in
66 ms, inf
```

```
2 #include <tensorflow/lite/micro/micro_error_reporter.h>
4 #include "tensorflow/lite/micro/micro_interpreter.h"

Output Serial Monitor X

Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/cu.usbmodem1101')

Standard-scaled model scores: 0.0000, 0.0000, 0.6055, 0.0000, 0.0000, 0.3945, 0.0000, 0.0000, 0.0000, 0.0000, 0.0
Min-max-scaled model scores: 0.0000, 0.0000, 0.0234, 0.0000, 0.0273, 0.0039, 0.0000, 0.9453, 0.0000, 0.0000, 0.00
Meta-classifier scores: 0.0625, 0.0000, 0.5078, 0.0898, 0.0508, 0.0195, 0.2031, 0.0078, 0.0000, 0.0156, 0.0391, 0
=====
Final prediction: Lying down
Class index: 2
Confidence score: 0.5078
=====

Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.0547, 0.0000, 0.0469, 0.0000, 0.0000, 0.0000, 0.0000, 0.3867, 0.4922, 0.0195
Standard-scaled model scores: 0.0000, 0.0039, 0.9727, 0.0000, 0.0000, 0.0000, 0.0000, 0.0039, 0.0078, 0.0000, 0.0039, 0.0
Min-max-scaled model scores: 0.0000, 0.0039, 0.1406, 0.0000, 0.1719, 0.0039, 0.6680, 0.0117, 0.0000, 0.0000, 0.00
Meta-classifier scores: 0.1250, 0.0000, 0.0781, 0.0859, 0.0039, 0.0000, 0.6406, 0.0078, 0.0000, 0.0078, 0.0391, 0
=====
Final prediction: Frontal elevation of arms
Class index: 6
Confidence score: 0.6406
=====

Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.0547, 0.0000, 0.0469, 0.0000, 0.0000, 0.0000, 0.0000, 0.3867, 0.4922, 0.0195
Standard-scaled model scores: 0.0000, 0.0039, 0.9688, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0156, 0.0000, 0.0039, 0.0
Min-max-scaled model scores: 0.0000, 0.0039, 0.1406, 0.0000, 0.1719, 0.0039, 0.6680, 0.0117, 0.0000, 0.0000, 0.00
Meta-classifier scores: 0.1289, 0.0000, 0.0781, 0.0820, 0.0039, 0.0000, 0.6445, 0.0078, 0.0000, 0.0078, 0.0391, 0
=====
Final prediction: Frontal elevation of arms
Class index: 6
Confidence score: 0.6445
=====

Samples collected: 25
```

```
Output Serial Monitor X
Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/cu.usbmodem1101')

Standard-scaled model scores: 0.0000, 0.0078, 0.9648, 0.0000, 0.0000, 0.0000, 0.0039, 0.0117, 0.0000, 0.0039, 0.0000, 0.0078
Min-max-scaled model scores: 0.0000, 0.0039, 0.1406, 0.0000, 0.1719, 0.0039, 0.6641, 0.0156, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.1211, 0.0000, 0.0625, 0.0859, 0.0039, 0.0000, 0.6562, 0.0078, 0.0000, 0.0078, 0.0430, 0.0039
-----
Final prediction: Frontal elevation of arms
Class index: 6
Confidence score: 0.6562
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.0195, 0.0000, 0.0078, 0.0000, 0.0000, 0.0000, 0.0000, 0.4258, 0.5391, 0.0078
Standard-scaled model scores: 0.0000, 0.0078, 0.9648, 0.0000, 0.0000, 0.0000, 0.0039, 0.0117, 0.0000, 0.0039, 0.0000, 0.0078
Min-max-scaled model scores: 0.0000, 0.0039, 0.1406, 0.0000, 0.1719, 0.0039, 0.6641, 0.0156, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.1211, 0.0000, 0.0625, 0.0859, 0.0039, 0.0000, 0.6562, 0.0078, 0.0000, 0.0078, 0.0430, 0.0039
-----
Final prediction: Frontal elevation of arms
Class index: 6
Confidence score: 0.6562
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.0000, 0.0273, 0.0000, 0.0117, 0.0000, 0.0000, 0.0000, 0.0000, 0.4180, 0.5312, 0.0117
Standard-scaled model scores: 0.0000, 0.0195, 0.7578, 0.0000, 0.0000, 0.0000, 0.0000, 0.0039, 0.0000, 0.0664, 0.0000, 0.1484
Min-max-scaled model scores: 0.0000, 0.0039, 0.1406, 0.0000, 0.1719, 0.0000, 0.6562, 0.0273, 0.0000, 0.0000, 0.0000, 0.0000
Meta-classifier scores: 0.1523, 0.0000, 0.0508, 0.1094, 0.0078, 0.0039, 0.5937, 0.0156, 0.0000, 0.0156, 0.0430, 0.0078
-----
Final prediction: Frontal elevation of arms
Class index: 6
Confidence score: 0.5937
-----
Samples collected: 25
Samples collected: 50
```

The output has 100 samples every 2 seconds

The predictions show that when I moved the Arduino around a lot it showed frontal arms raised but when I let it rest on the table it showed lying down. This is very intuitive and shows that although the model is not perfect it shows it can classify certain movements with meaningful accuracy.

Some issues were unstable predictions during motion. The model became much less confident when dealing with motion. Also, the standard and min-max scalers were fit on mHealth statistics. Live BMI270 data may fall outside those ranges, giving the scaled branches out-of-distribution inputs. In terms of practical improvements I would recollect the training data from actual devices or at least some data for some post-training. We could also add a confidence threshold at the application stage just to only output a result if it is beyond a certain threshold to reduce confusion.

#### Question 4:

Yes often the board predicted frontal arms raised for very different movements such as jerky strokes or more slow twists. However when at rest the model was quite confident in classifying as lying down. This could happen for many reasons but a main culprit is training

and deployment mismatch. The model might be outputting the frontal elevation class because the environment or output of the sensors on the Arduino are the most similar and not because the actual movement is similar. The other issue is that I have a very short cord so I couldn't produce very dynamic range of motion. The main reason I think is that when the model wasn't very confident it still just outputs the highest class even if it isn't necessarily a strong match.